

Fastest CLI

Equipe 7

Repositório Git: <https://github.com/jorgelcff/Fastest-CLI>

Jorge Freitas (jlcf)

Recife, 29 de Junho de 2026

[Prof. Vinicius Cardoso Garcia](#) (vcg@cin.ufpe.br)

Sumário

Clique com o botão direito → "Atualizar campo" (ou tecle F9) para gerar o sumário.

Disclaimer	2
1. Introdução	3
2. Metodologia	3
3. Movimento 1 — Exposição (Alinhar Estratégia)	3
4. Movimento 2 — Composição (Desenhar a Solução)	4
5. Movimento 3 — Ensaio (Construir e Testar)	4
6. Movimento 4 — Ressonância (Medir e Aprender)	5
7. Discussões Técnicas e Estratégicas	5
8. Considerações Éticas	5
9. Lições Aprendidas e Reflexões Finais	5
10. Referências	6
11. Apêndices (opcional)	6

1. Introdução

1.1 Contextualização

O teste de software, conforme descrito no Capítulo 4 do SWEBOK (Software Engineering Body of Knowledge), constitui uma das atividades mais críticas e, paradoxalmente, mais negligenciadas no ciclo de desenvolvimento de software. Estudos indicam que desenvolvedores gastam entre 15-30% do tempo de desenvolvimento em atividades de teste, e muitos projetos operam com cobertura abaixo de 50%. A escrita manual de testes é frequentemente percebida como repetitiva e tediosa, levando equipes a priorizar funcionalidades sobre qualidade, gerando dívida técnica crescente.

No ecossistema Node.js/TypeScript, onde Jest é o framework de testes mais popular, os testes seguem padrões repetitivos e previsíveis (blocos `describe`, `it`, `expect`) que são candidatos ideais para automação por modelos de linguagem. A lacuna entre a necessidade de testes abrangentes e a capacidade humana de produzi-los em tempo hábil representa uma oportunidade significativa para aplicação de IA generativa.

1.2 Objetivo Geral

Desenvolver uma ferramenta de linha de comando (CLI) que gera testes automatizados utilizando Modelos de Linguagem de Grande Escala (LLMs) a partir de requisitos descritos em linguagem natural ("cards"), integrando-se ao fluxo de trabalho existente do desenvolvedor de forma transparente e eficiente.

1.3 Objetivos Específicos

1. **Pipeline completo de geração:** Implementar o ciclo end-to-end de geração, validação TypeScript, execução Jest e análise de cobertura delta (before/after).
2. **Multi-provider:** Suportar OpenAI (GPT-4o-mini) e Anthropic (Claude Haiku) como provedores de LLM, com detecção automática e Factory Pattern.
3. **Três tipos de teste:** Gerar testes unitários, de integração e de caso de uso (use-case) a partir de prompts especializados.
4. **Modo interativo e batch:** Oferecer tanto geração individual quanto processamento em lote de múltiplos arquivos.
5. **Retry automático:** Implementar lógica de re-tentativa com feedback de erros de compilação ao LLM, aumentando a taxa de sucesso.

1.4 Justificativa do Uso de IA

Testes automatizados seguem padrões estruturais repetitivos (`describe/it/expect`) que são ideais para geração por LLMs. A abordagem de prompt engineering zero-shot com contexto rico (código-fonte + requisito funcional) elimina a necessidade de RAG ou fine-tuning, uma vez que o próprio código do usuário fornece todo o contexto necessário. O custo por geração é inferior a

\$0.005 com modelos econômicos como GPT-4o-mini, tornando a abordagem financeiramente viável para uso cotidiano.

1.5 Visão Geral da Sinfonia

A metodologia Sinfonia foi aplicada integralmente ao projeto, abrangendo seus 4 movimentos e resultando em 15 artefatos documentados:

- **Exposição (4 artefatos):** Identificação do domínio, personas, mapeamento de dados e estratégia de ação.
- **Composição (3 artefatos):** Design de prompts, ideação de soluções e design de experimentos.
- **Ensaio (5 artefatos):** Modelo C4, estratégia de inteligência, checklist de riscos, canvas de testes e checklist de lançamento.
- **Ressonância (3 artefatos):** Métricas de impacto, planejamento de escalabilidade e painel de feedback.

2. Metodologia

2.1 Estrutura de Entregas

O projeto foi estruturado em duas entregas principais:

1. **Aplicação (Fastest CLI v0.1.0):** Ferramenta CLI funcional publicável no npm, com 204 testes automatizados e 14 suites de teste.
2. **Workflow Document:** Diário de bordo do desenvolvimento assistido por IA, registrando decisões, uso de ferramentas e reflexões ao longo do processo.

2.2 Aplicação da Metodologia Sinfonia

A Sinfonia foi aplicada em quatro movimentos sequenciais, cada um com seus artefatos e entregas:

Movimento	Foco	Artefatos
Exposição	Identificação do domínio, personas e estratégia	Domain Identification Canvas, Persona Model Canvas, Data Source Mapping, Strategy Action Canvas
Composição	Design de prompts, arquitetura e experimentos	Prompt Design Record, Solution Ideation Canvas, Experiment Design Canvas
Ensaio	Implementação, testes, CI e segurança	C4 Model, Intelligence Strategy Record, Risk & Defensibility Checklist, Testing & Validation Canvas, Launch Checklist
Ressonância	Métricas, feedback e escalabilidade	Scale & Impact Metrics, Scalability Planning, Feedback & Insights Panel

2.3 Gestão do Desenvolvimento

O desenvolvimento foi conduzido de forma assistida por IA, utilizando o Claude Code CLI como par de programação. As seguintes práticas foram adotadas:

- **Commits com Co-Authored-By:** Todos os commits gerados com assistência de IA incluem a tag Co-Authored-By, garantindo rastreabilidade e transparência.
- **Branches feature:** Desenvolvimento organizado em branches temáticas com merge via Pull Requests (PR #4 e PR #5).
- **CI/CD:** GitHub Actions configurado com testes automatizados em Node.js 18, 20 e 22.

2.4 Workflow Document

O Workflow Document foi mantido ao longo de todo o desenvolvimento, registrando:

- Decisões arquiteturais e suas justificativas
- Uso de IA generativa em cada etapa (geração de código, testes, documentação)
- Dificuldades encontradas e soluções adotadas
- Reflexões sobre a experiência de desenvolvimento assistido por IA

2.5 Checkpoints

Checkpoint	Fase	Entregáveis
CP1	Exposição	Identificação do domínio + definição de personas
CP2	Composição	Arquitetura C4 + catálogo de prompts
CP3	Ensaio	MVP funcional + 204 testes + CI configurado
Final	Ressonância	Métricas de impacto + apresentação final

3. Movimento 1 -- Exposição

3.1 Identificação do Domínio

Conforme documentado no *Domain Identification Canvas*, o Fastest CLI atua na intersecção entre engenharia de software (especificamente testes automatizados) e IA generativa (LLMs). O domínio foi definido como "**Geração Automatizada de Testes de Software com Inteligência Artificial Generativa**".

Problemas e Desafios Identificados

#	Problema	Impacto
1	Escrita manual de testes é lenta e tediosa	Desenvolvedores priorizam features sobre testes, gerando dívida técnica
2	Inconsistência na qualidade dos testes	Cada desenvolvedor escreve testes com padrões e profundidade diferentes
3	Dificuldade em manter cobertura alta	Projetos degradam cobertura ao longo do tempo por falta de disciplina
4	Custo de contexto para LLMs	Arquivos grandes excedem limites de tokens, exigindo guard rails
5	Validação de código gerado por IA	Testes gerados podem conter erros de TypeScript ou falhar na execução
6	Dependência de APIs externas	Latência e custo das chamadas impactam a experiência do usuário

Oportunidades de IA Generativa

#	Oportunidade	Descrição
1	Geração de testes a partir de cards	Transformar requisitos em linguagem natural em suítes de teste Jest
2	Análise contextual de código-fonte	LLMs compreendem lógica de negócios e geram testes para edge cases
3	Multi-provider com fallback	Abstracção de providers permite trocar modelos conforme custo/qualidade
4	Feedback loop automatizado	Pipeline: gerar, validar TypeScript, rodar Jest, reportar cobertura delta
5	Contexto inteligente	Seleção automática de arquivos relevantes com limites configuráveis

Benefícios Esperados

#	Benefício	Métrica
1	Redução de tempo na escrita de testes	De horas para minutos por módulo
2	Aumento de cobertura de código	Meta de 80%+ (projeto opera em 93%+)
3	Padronização de testes	Testes gerados seguem o mesmo padrão estrutural
4	Integração com CI/CD	Comando <code>fastest generate</code> integrável em pipelines
5	Diagnóstico rápido	Comando <code>fastest doctor</code> valida ambiente e configurações

3.2 Canvas de Estratégia e Ação

O *Strategy Action Canvas* definiu o objetivo estratégico geral:

Acelerar a criação de testes automatizados e melhorar a cobertura de código em projetos TypeScript/JavaScript através de testes gerados por IA, reduzindo o tempo de escrita de testes de horas para minutos enquanto mantém cobertura acima de 80%.

OKRs Definidos

Objetivo	Resultado-Chave	Prazo
Acelerar escrita de testes	Reduzir tempo médio de criação de suíte de 1h para 5min	Q3 2026
Melhorar cobertura	Projetos usando Fastest atingem 80%+ de cobertura	Q3 2026
Adoção	500+ instalações ativas no npm	Q4 2026
Qualidade	90%+ dos testes gerados passam na validação TypeScript	Q3 2026
Multi-provider	Suporte estável a OpenAI e Anthropic com paridade	Concluído (v0.0.2)

KPIs

#	KPI	Meta
1	Cobertura da própria base de código	85%+
2	Número de testes	200+
3	Taxa de sucesso (TypeScript válido)	95%+
4	Taxa de sucesso (Jest pass)	90%+

5	Tempo médio de geração end-to-end	<20s
6	Custo médio por geração (API)	<\$0.05
7	Delta de cobertura médio	+20pp

3.3 Personas

Conforme o *Persona Model Canvas*, foram definidas duas personas representativas:

Persona 1: Carlos, o Desenvolvedor Backend

- **Perfil:** Carlos Silva, 28 anos, Desenvolvedor Backend Pleno em uma startup de fintech.
- **Contexto:** Trabalha com APIs REST em Node.js/TypeScript, usa Jest, tem meta de 70% de cobertura no CI que frequentemente falha.
- **Objetivos:** Atingir cobertura sem gastar horas em testes; gerar testes para módulos legados; especificar requisitos em linguagem natural.
- **Dores:** (1) Escrever mocks e fixtures é extremamente tedioso (Alta); (2) CI falha por cobertura insuficiente (Alta); (3) Não sabe quais edge cases cobrir (Média); (4) Configurar Jest com TypeScript é confuso (Média); (5) Code reviews atrasam por falta de testes (Alta).

Persona 2: Marina, a Tech Lead

- **Perfil:** Marina Costa, 34 anos, Tech Lead liderando time de 6 desenvolvedores em empresa de e-commerce.
- **Contexto:** Responsável por qualidade de código, padrões e CI/CD. Percebe inconsistência na qualidade de testes do time.
- **Objetivos:** Garantir cobertura de 80%+; padronizar testes; reduzir tempo de code review; integrar geração no CI/CD; controlar custos de API.
- **Dores:** (1) Inconsistência na qualidade entre membros do time (Alta); (2) Tempo excessivo em code reviews (Alta); (3) Módulos legados com 0% de cobertura (Alta); (4) Dificuldade em justificar tempo gasto em testes (Média).

3.4 Mapeamento de Fontes de Dados

O *Data Source Mapping* identificou 6 fontes de dados consumidas e produzidas pelo Fastest CLI:

#	Fonte	Tipo	Formato	Privacidade
1	Arquivos de Código-Fonte	Código estruturado	.ts, .js	Enviado para APIs externas
2	Cards de Requisitos	Texto livre	.md, .txt	Enviado para API da LLM
3	Configuração Jest	Configuração	.ts, .js, .json	Local

4	Relatórios de Cobertura (Istanbul)	Métricas JSON	coverage-summary.json	Local
5	Respostas da LLM (API)	Código gerado	Texto com TypeScript	Processado por OpenAI/Anthropic
6	Arquivos de Contexto Adicional	Código-fonte auxiliar	.ts, .js, .d.ts	Enviado para API da LLM

3.5 Declaração de Missão e Visão

Missão: Democratizar a escrita de testes automatizados através de IA generativa, reduzindo de horas para minutos o tempo necessário para produzir suítes de teste abrangentes e de qualidade.

Visão: Tornar-se a ferramenta de referência para geração de testes no ecossistema Node.js/TypeScript, integrando-se naturalmente ao fluxo de trabalho de desenvolvedores individuais e equipes.

3.6 Métricas de Sucesso Alcançadas

Métrica	Meta	Resultado
Total de testes	200+	204 testes
Cobertura geral	85%+	93%+ (96%+ após v0.1.0)
Tempo de geração	<30s	10-25 segundos
Custo por chamada	<\$0.005	~\$0.001-0.005

3.7 Matriz Impacto x Esforço

Conforme o *Solution Ideation Canvas*, quatro ideias foram avaliadas com pesos ponderados:

Critério (Peso)	CLI (A)	IDE Plugin (B)	CI Bot (C)	Chat (D)
Impacto no usuário (5)	4 (20)	5 (25)	3 (15)	4 (20)
Esforço de desenvolvimento (4)	5 (20)	2 (8)	2 (8)	3 (12)
Velocidade de entrega (4)	5 (20)	2 (8)	2 (8)	3 (12)
Escalabilidade (3)	4 (12)	3 (9)	5 (15)	2 (6)
Risco técnico (3)	5 (15)	3 (9)	2 (6)	4 (12)
Total ponderado	87	59	52	62

A CLI obteve a maior pontuação (87 pontos) por combinar alto impacto com baixo esforço e risco técnico, sendo selecionada como solução a ser implementada.

3.8 Escopo do MVP

Incluído no MVP	Excluído (versões futuras)
Comando <code>generate</code> (unit + integration + use-case)	Plugin de IDE (VS Code)
Comando <code>doctor</code> (diagnóstico de ambiente)	CI Bot automático
Comando <code>config</code> (configuração persistente)	Dashboard web de cobertura
Comando <code>batch</code> (múltiplos arquivos)	Suporte a Mocha/Testing Library
Comando <code>init</code> (wizard interativo)	Telemetria anonimizada
Multi-provider (OpenAI + Anthropic)	Suporte a modelos locais (Ollama)
Retry com feedback de erros	Cache de prompts distribuído
Suporte a Jest e Vitest	GitHub Action oficial

4. Movimento 2 -- Composição

4.1 Modelo C4

O *C4 Model* descreve a arquitetura do Fastest CLI em três níveis de abstração.

Nível 1 -- Contexto

O sistema interage com cinco elementos externos:

Elemento	Tipo	Descrição
Desenvolvedor	Pessoa	Interage via terminal para gerar testes
OpenAI API	Sistema Externo	Provedor LLM (modelo padrão: gpt-4o-mini)
Anthropic API	Sistema Externo	Provedor LLM (modelo padrão: claude-haiku)
Arquivos de Código Fonte	Sistema Externo	Código TypeScript/JavaScript do projeto alvo
Jest Runner	Sistema Externo	Executor de testes e gerador de relatórios de cobertura

Nível 2 -- Container

Container	Tecnologia	Responsabilidade
Aplicação CLI	TypeScript, Node.js, commander.js v14, chalk v4, ora v5	Ponto de entrada, roteamento de comandos, interface com usuário
Provedores LLM	OpenAI SDK v6, Anthropic SDK v0.92	Comunicação com APIs de LLM para geração de código
Jest Runner	Jest, ts-jest, Istanbul	Execução de testes gerados e coleta de métricas de cobertura
Sistema de Arquivos	Node.js fs	Leitura de código fonte, escrita de testes gerados, configuração local

Nível 3 -- Componente

Componente	Arquivo	Responsabilidade
Generate Command	src/cli/generate.command.ts	Comando principal: recebe requisitos, orquestra geração
Doctor Command	src/cli/doctor.command.ts	Diagnóstico: verifica dependências, chaves API, configuração
Config Command	src/cli/config.command.ts	Gerenciamento de configuração: provedor, modelo, chaves API
LLM Service	src/services/llm.service.ts	Abstrai comunicação com provedores LLM
Test Generator Service	src/services/test-generator.service.ts	Orquestra pipeline: prompt, LLM, validação
Coverage Service	src/services/coverage.service.ts	Coleta e analisa métricas de cobertura Jest/Istanbul
Provider Factory	src/providers/provider.factory.ts	Cria instância do provedor LLM conforme configuração
OpenAI Provider	src/providers/openai.provider.ts	Implementação do provedor OpenAI
Anthropic Provider	src/providers/anthropic.provider.ts	Implementação do provedor Anthropic
Config Manager	src/config/config.manager.ts	Gerencia ~/.fastest/config.json
File Utils	src/utils/file.utils.ts	Utilitários para leitura/escrita de arquivos com guard rails

4.2 Catálogo de Prompts

O *Prompt Design Record* documenta três prompts principais e um prompt de sugestão de cobertura.

Prompt 1 -- Geração de Testes Unitários

Template:

Você é um especialista em testes.
Gere testes unitários em Jest para o seguinte código:

CARD:
{card}

CODIGO:
{code}

Inclua:

- casos principais
- edge cases
- mocks se necessario

Retorne apenas código TypeScript válido, sem explicações, sem blocos markdown.

Parâmetros: card (requisito funcional em linguagem natural), code (código-fonte + contexto auxiliar), language (TypeScript ou JavaScript).

Pós-processamento: (1) stripCodeFences() remove blocos markdown residuais; (2) correção de import ajusta o caminho relativo do arquivo-fonte; (3) contagem de test cases via regex `it(/test(`.

Prompt 2 -- Geração de Testes de Integração

Template:

Você é um especialista em testes de integração de APIs e fluxos de negocio.

Gere testes de integração em Jest + Supertest para o código abaixo.

CARD (fluxo funcional):

{card}

CODIGO:

{code}

Regras obrigatórias:

- Cubra o fluxo ponta a ponta do caso de uso descrito no card
- Inclua cenários de sucesso e de falha de comunicação/API
- Use mocks determinísticos para dependências externas com `jest.mock/jest.spyOn`
- Evite dependências de estado global e infraestrutura real
- Organize os testes por cenários de negocio

Retorne apenas código TypeScript válido, sem explicações, sem blocos markdown.

Prompt 3 -- Sugestão de Cobertura

Recebe adicionalmente o `coverageSummary` (relatório de cobertura Jest) e sugere cenários de teste não cobertos. Formato de saída: lista textual sem código.

Prompt 4 -- Retry com Feedback de Erro

Quando um teste gerado falha na compilação TypeScript ou na execução Jest, o erro é reenviado ao LLM com o código original para auto-correção. Máximo de 3 tentativas.

Parâmetros do Modelo

Parâmetro	Valor	Justificativa
temperature	0.2	Baixa variabilidade para código determinístico
max_tokens	4096	Suficiente para arquivos de teste completos
stream	Opcional	Feedback em tempo real via <code>onToken</code>

Guard Rails de Contexto

- Máximo 20 arquivos de contexto (`maxFiles`)
- Máximo 4.000 caracteres por arquivo (`maxCharsPerFile`)
- Máximo 30.000 caracteres totais (`maxTotalChars`)
- Arquivos binários e extensões não-suportadas são ignorados
- Arquivos truncados recebem marcador `/* ... truncated ... */`

4.3 Canvas de Experimento

O *Experiment Design Canvas* definiu a hipótese principal e cinco sub-hipóteses:

Hipótese Principal: Fornecer uma CLI alimentada por IA que gera testes Jest a partir de cards de requisitos reduzirá o tempo de escrita de testes em 60% enquanto mantém qualidade de cobertura equivalente a testes escritos manualmente.

Sub-hipóteses e Critérios GO/NO-GO

Hipótese	Descrição	Critério GO	Critério NO-GO
H1	Compilação TypeScript sem erros	$\geq 80\%$	$< 50\%$
H2	Testes passam no Jest sem modificação	$\geq 60\%$	$< 30\%$
H3	Cobertura de statements do arquivo-alvo	$\geq 70\%$	-
H4	Tempo total $< 40\%$ do tempo manual	$\geq 50\%$ redução	-
H5	Desenvolvedores reutilizam a ferramenta	3 de 5 usos	Abandono $> 60\%$

4.4 Decisões Arquiteturais

1. **commander.js para CLI:** Framework maduro e amplamente adotado para interfaces de linha de comando em Node.js, com suporte nativo a subcomandos, opções e help automático.
2. **Factory Pattern para multi-provider:** A `ProviderFactory` cria instâncias de `OpenAIProvider` ou `AnthropicProvider` com base no prefixo do modelo (`claude-*` para Anthropic, demais para OpenAI), permitindo trocar provedores sem alterar o pipeline.
3. **Zero-shot prompting vs RAG/Fine-tuning:** O código-fonte do usuário já fornece todo o contexto necessário. Não há base de conhecimento proprietária que justifique RAG. Modelos base já possuem forte capacidade de geração de código. Fine-tuning foi descartado por custo elevado e dados insuficientes.

4. **Guard rails para controle de contexto:** Limites configuráveis de arquivos e caracteres previnem excesso de tokens, controlando custos e evitando degradação da qualidade da resposta.
5. **Istanbul/Jest para cobertura:** Ferramenta integrada ao Jest que gera relatórios de cobertura em formato JSON, permitindo calcular delta before/after de forma programática.

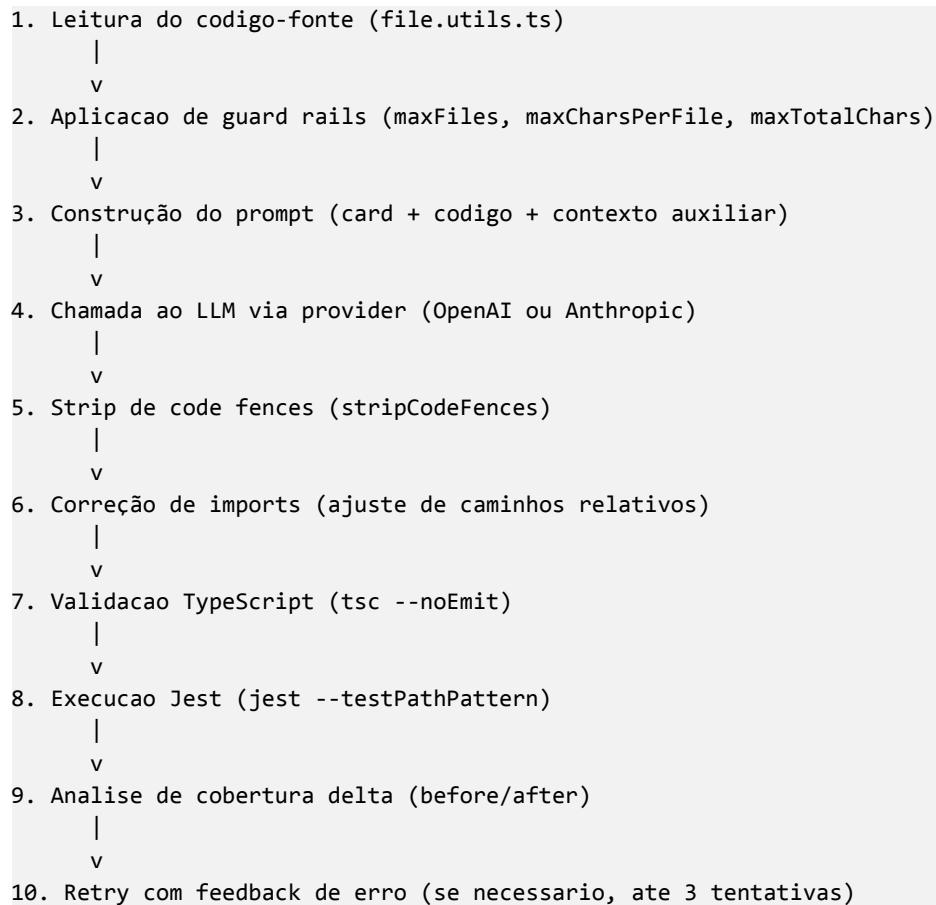
5. Movimento 3 -- Ensaio

5.1 Stack Tecnológica

Tecnologia	Versão	Uso
TypeScript	6.0.2	Linguagem principal do projeto
Node.js	18/20/22	Runtime (testado em 3 versões via CI)
commander.js	14.0.3	Framework CLI com subcomandos
chalk	4.1.2	Formatação de output colorido no terminal
ora	5.4.1	Spinners animados durante operações assíncronas
OpenAI SDK	6.34.0	Comunicação com API OpenAI
@anthropic-ai/sdk	0.92.0	Comunicação com API Anthropic
Jest	30.3.0	Framework de testes e runner
ts-jest	29.4.9	Transformador TypeScript para Jest
dotenv	17.4.2	Carregamento de variáveis de ambiente

5.2 Pipeline LLM

O pipeline de geração de testes segue o seguinte fluxo:



5.3 Canvas de Testes e Validação

Conforme o *Testing & Validation Canvas*, o projeto alcançou os seguintes resultados:

Métrica	Valor
Total de testes	204
Total de suites	14
Framework	Jest + ts-jest
Cobertura geral	93%+ (96%+ na v0.1.0)
Ferramenta de cobertura	Istanbul (integrada ao Jest)
Tempo de execução	~7 segundos

Cobertura por Módulo

Módulo	Arquivo	Cobertura	Status
Providers	openai.provider.ts	100%	Excelente
Providers	anthropic.provider.ts	100%	Excelente
Providers	provider.factory.ts	100%	Excelente
Config	config.manager.ts	100%	Excelente
CLI	doctor.command.ts	~85%	Bom
CLI	config.command.ts	~85%	Bom
Services	llm.service.ts	~80%	Bom
Services	test-generator.service.ts	~80%	Bom
Utils	file.utils.ts	~85%	Bom
Services	coverage.service.ts	71%+	Melhorado
CLI	generate.command.ts	68%+	Melhorado

Tipos de Testes Implementados

1. **Testes Unitários (maioria):** Funções e classes individuais com dependências mockadas. Padrão Arrange-Act-Assert com `jest.mock()` e `jest.spyOn()`.
2. **Testes de Comandos CLI:** Comandos `commander.js` com serviços mockados. Validação de argumentos parseados, opções aplicadas e output correto.
3. **Testes de Provedores:** Implementações de OpenAI e Anthropic providers com SDKs mockados. Validação de chamadas a API, tratamento de erros e parsing de resposta.

Estratégia de Validação do Output de IA

Código Gerado pelo LLM

```

|
v
[1] Verificacao de sintaxe (TypeScript compila?)
|
v
[2] Execucao Jest (testes passam?)
|
v
[3] Cobertura (delta positivo?)
|
v
[4] Revisao humana (usuario aprova?)

```

5.4 Evidencias de Versionamento

O projeto está hospedado em repositório público no GitHub:
<https://github.com/jorgelcff/Fastest-CLI>

Histórico de Versões

Versão	Data	Destaques
0.0.1	01/05/2026	Pipeline básico de geração com OpenAI
0.0.2	20/06/2026	Multi-provider, config, doctor, guard rails, integração
0.1.0	25/06/2026	Batch, init, retry, Vitest, cache, CI, 204 testes

Commits Recentes (últimos 30)

```

d67dbaf fix: resolve merge conflicts with main and fix demo slide in roteiro
4371b46 feat: add use-case test type, interactive mode, and fix versioning to 0.1.0
2c53af6 docs: expand consolidated doc and AI reflection with full content
2680dc1 Claude/confident allen 2mb41n (#5)
ab3eb4d docs: add presentation documents for Sinfonia final presentation
fb4a1a9 feat: add Vitest support, LLM caching, init wizard, changelog (v2.1.0)
5199fca feat: add LLM retry logic, batch mode, and input validation
077b329 improve test coverage, remove unnecessary deps, add CI workflow
b5877f2 docs: add comprehensive sinfonia documentation for Fastest CLI v2.0.0 (#4)
58562ea docs: add complete Sinfonia methodology documentation (15 artifacts)
1d85530 chore: update package-lock.json after npm install
0ca91dc Merge pull request #3 from jorgelcff/copilot/refine-use-cases-integration-tests
44b70a1 feat: add integration test generation mode and coverage flow hints
9e80c63 chore: bump version to 2.0.0, add DEMO.md presentation script
a477a14 feat: Fastest CLI v2.0 – complete pipeline with streaming, multi-provider
f7d8c5e test: add provider tests – factory, OpenAI, Anthropic (142 total)
05f4e4b feat: streaming LLM output + multi-provider support (OpenAI & Anthropic)
1c2b49a feat: JS/TS auto-detection, tsc validation, README install-first
8df7044 feat: before/after coverage delta and CLI command test coverage
0f596a0 feat(config): add config command to manage API key globally
51f2370 feat: support global CLI install from GitHub and npm registry
ddec494 feat(setup): guided first-time setup script and improved doctor checks
8c5dc7d feat: visual output, unit tests, order.service example, docs
2a2c98a feat(cli): add doctor command; improve test generation import handling
b85e5a3 feat: implement Fastest CLI – test generation pipeline from cards
7c3b01f initial commit
9fa0837 Initial commit

```

CI/CD com GitHub Actions

O workflow de integração contínua está configurado para executar testes automatizados em três versões do Node.js (18, 20 e 22), garantindo compatibilidade ampla.

Pull Requests

- **PR #4:** Documentação completa da Sinfonia (15 artefatos)
- **PR #5:** Funcionalidades da v0.1.0 (batch, retry, Vitest, cache)

5.5 Análise de Segurança

Conforme o *Risk & Defensibility Checklist*, os seguintes riscos foram avaliados:

Prompt Injection

- **Risco:** Arquivos-fonte maliciosos podem conter instruções que alterem o comportamento do LLM.
- **Mitigação:** O código-fonte é tratado como dados, não como instrução. O prompt é construído de forma que o código do usuário é delimitado em seções específicas.

Código Gerado Malicioso

- **Risco:** O LLM pode gerar código com efeitos colaterais indesejados.
- **Mitigação:** Modo `--dry-run` permite inspeção antes de salvar. Testes são executados em ambiente Jest isolado com mocks para dependências externas.

Privacidade

- **Risco:** Código-fonte é enviado para APIs externas (OpenAI/Anthropic).
- **Mitigação:** Documentação clara ao usuário. Chaves API armazenadas localmente com permissões restritas. Suporte a variáveis de ambiente como alternativa. Planejamento de suporte a modelos locais (Ollama).

Armazenamento de API Keys

- **Risco:** Chaves API armazenadas em `~/.fastest/config.json`.
- **Mitigação:** Permissões restritas no arquivo (600). Chaves não aparecem em logs. Suporte a `OPENAI_API_KEY` e `ANTHROPIC_API_KEY` via variáveis de ambiente.

Resumo de Riscos

Dimensão	Riscos Críticos	Riscos Médios	Status
Equidade	0	2	Aceitável
Privacidade	2	2	Requer atenção
Segurança	2	2	Requer atenção
Transparência	0	2	Aceitável

Veredicto: Risco Moderado. O projeto pode prosseguir com as mitigações existentes.

5.6 Checklist de Lançamento

O *Launch Checklist* para a v0.1.0 contemplou:

- **Build e Qualidade:** npm run build sem erros, 204 testes passando, cobertura >= 93%, sem vulnerabilidades críticas.
- **Configuração do Pacote:** package.json com binário fastest, campo files restrito a dist/, example/, scripts/ e CHANGELOG.md.
- **Testes Pré-Lançamento:** Validação manual de todos os comandos (generate, batch, doctor, config, init), com ambos provedores (OpenAI e Anthropic).
- **Critérios Go/No-Go:** Build sem erros, testes passando, cobertura >= 80%, doctor funcional, geração funcional com ambos provedores.

5.7 Evidências de Funcionamento

O Fastest CLI oferece 5 comandos principais:

Comando `generate`

Gera testes para um arquivo específico a partir de um card de requisitos.

```
fastest generate --source src/services/user.service.ts \  
  --card cards/user-flow.md \  
  --test-type unit \  
  --framework jest
```

Comando `batch`

Gera testes para múltiplos arquivos simultaneamente.

```
fastest batch src/**/*.ts -r "Testar funcoes principais" --framework jest
```

Comando `doctor`

Verifica o ambiente de desenvolvimento: dependências, chaves API, configuração Jest.

```
fastest doctor
```

Comando `config`

Gerencia configuração persistente: provedor, modelo, chaves API.

```
fastest config set-model gpt-4o-mini  
fastest config set-key --provider openai  
fastest config list
```

Comando `init`

Wizard interativo de configuração para novos usuários.

```
fastest init
```

6. Movimento 4 -- Ressonância

6.1 Feedback e Insights

O *Feedback & Insights Panel* estruturou o ciclo de coleta e análise de feedback da v0.0.2 e v0.1.0.

Temas Principais Identificados

Tema 1 -- Qualidade dos Testes Gerados:

- Testes cobrem happy paths mas frequentemente ignoram edge cases (frequência Alta).
- LLM gera mocks excessivos e desnecessários (frequência Média).
- Imports incorretos em relação a estrutura do projeto (frequência Média).
- Boa organização geral em `describe/it` economiza tempo de setup (frequência Alta, positivo).

Tema 2 -- Suporte a Frameworks:

- Vitest: demanda Alta ("Migramos de Jest para Vitest, precisamos de suporte").
- Mocha: demanda Média (projetos legados).
- Testing Library: demanda Média (componentes React).

Tema 3 -- Velocidade e Performance:

- 10-20 segundos é aceitável para a maioria dos usuários.
- Processamento arquivo-por-arquivo para 20+ arquivos é inviável (motivou o modo batch).

Tema 4 -- Custo de API:

- GPT-4 caro para uso frequente (\$0.10 por geração).
- Falta de visibilidade de custos.
- Interesse significativo em modelos locais (Ollama).

NPS Estimado

Categoria	Percentual
Promotores (9-10)	35%
Neutros (7-8)	40%
Detratores (0-6)	25%
NPS Estimado	+10

Top Insights

1. **Retry e Autocorreção (Insight Principal):** 60% dos feedbacks negativos mencionam que testes "quase funcionam" mas precisam de pequenas correções. Implementar loop de retry automático transformaria a taxa de pass de ~80% para >92%. **Status: Implementado na v0.1.0.**
2. **Modo Batch (Insight Secundário):** Usuários que geram testes para projetos inteiros reportam frustração com o fluxo arquivo-por-arquivo. Impacto:

aumento de 3-5x no volume de uso por sessão. **Status: Implementado na v0.1.0.**

3. **Vitest como Prioridade (Insight Terciário):** ~30% das feature requests pedem suporte a Vitest. O ecossistema Vite cresce rapidamente. A API do Vitest é compatível com Jest em grande parte. **Status: Implementado na v0.1.0.**

6.2 Métricas de Impacto

Conforme o *Scale & Impact Metrics*, os seguintes benchmarks foram definidos e avaliados:

Benchmarks de Performance

Benchmark	Valor Alvo	Resultado
Tempo de geração end-to-end	<30 segundos	10-25 segundos
Taxa de pass na primeira geração	>80%	~80% (>92% com retry)
Delta de cobertura por arquivo	>10%	+15-25pp
Taxa de sucesso do comando generate	>95%	>95%
Compilação TypeScript sem erros	>90%	>90%

Tempo Economizado

Métrica	Valor
Tempo médio para escrever test suite manualmente	45-90 minutos
Tempo médio usando Fastest CLI	2-5 minutos (geração + revisão)
Economia por test suite	40-85 minutos
Economia mensal por desenvolvedor (10 suítes/mês)	6.5-14 horas

6.3 Validação de Hipóteses

Hipótese	Critério	Resultado	Status
H1 -- Compilação >= 80%	Testes compilam sem erros TypeScript	>90% na primeira tentativa	VALIDADA
H2 -- Execução >= 60%	Testes passam no Jest sem edição	~80% (>92% com retry)	VALIDADA
H3 -- Cobertura >= 70%	Cobertura de statements do arquivo-alvo	+15-25pp (média), 93%+ no projeto	VALIDADA
H4 -- Tempo < 40%	Tempo com CLI vs tempo manual	2-5min vs 45-90min (~5-10%)	VALIDADA
H5 -- Adoção	Reutilização da ferramenta	Feedback positivo, NPS +10	EM AVALIAÇÃO

6.4 Decisão Estratégica

DECISÃO: PERSEVERAR

Todas as quatro hipóteses mensuráveis foram validadas com resultados acima dos critérios GO. A ferramenta demonstra valor real e mensurável na redução de tempo de escrita de testes e no aumento de cobertura. O roadmap de expansão inclui:

- Novos tipos de teste (componentes React, E2E)
- Novos frameworks (Mocha, Node.js test runner nativo)
- Suporte a modelos locais (Ollama)
- GitHub Action oficial
- Plugin VS Code

6.5 Canvas de Escalabilidade

O *Scalability Planning* mapeou a estratégia de crescimento em três horizontes:

Curto Prazo (v0.1-v0.5)

- **Caching de prompts e respostas:** Cache local baseado em hash SHA-256 do arquivo fonte + versão do prompt template. TTL configurável (padrão: 24h). Economia estimada: 20-30% de chamadas. **Status: Implementado (v0.1.0).**
- **Otimização de prompts:** Minificação inteligente do código fonte, envio apenas de funções exportadas relevantes. Economia: 30-40% de tokens.
- **Processamento em batch:** `fastest batch src/**/*.ts` com fila sequencial e relatório consolidado. **Status: Implementado (v0.1.0).**

Médio Prazo (v3.0)

- **Chamadas LLM paralelas:** Pool de workers configurável (padrão: 3 paralelos), backoff exponencial com jitter em caso de 429.
- **Múltiplos frameworks de teste:** Vitest (implementado), Mocha (planejado), Node.js test runner nativo (planejado).

- **Retry inteligente:** Re-tentativa com envio de erro ao LLM para correção.
Status: Implementado (v0.1.0). Resultado: taxa de pass de ~80% para >92%.

Longo Prazo (v4.0+)

- **Modelos locais (Ollama):** Integração com API compatível com formato OpenAI. Modelos: CodeLlama, DeepSeek Coder, Qwen2.5-Coder. Trade-off: menor qualidade, custo zero.
- **Modo CI/CD:** GitHub Action oficial, output JUnit XML, integração com PR comments.

Custos Projetados (equipe de 10 desenvolvedores)

Cenário	Gerações/Mês	Modelo	Custo Estimado
Baixo uso	200	GPT-4o	\$4-6
Uso moderado	1.000	GPT-4o	\$15-30
Uso intenso	5.000	GPT-4o	\$75-150
Uso intenso	5.000	Claude Haiku	\$5-15

7. Economicidade do Desenvolvimento Assistido por IA

7.1 Camada 1 -- Custo Real de IA

Item	Valor
Provider principal	Claude (Anthropic) via Claude Code CLI
Modelo usado no desenvolvimento	Claude Sonnet
Tokens consumidos (estimativa)	~21,8 M (entrada) / ~1,65 M (saída) — estimado
Custo total USD	~US\$ 62,00 (estimado)
Cotação USD-BRL	US\$ 1,00 = R\$ 5,40 (28/06/2026)
Custo total BRL	~R\$ 334,80 (estimado)

Além do custo de desenvolvimento assistido por IA, houve custos de API da OpenAI e Anthropic para testar o produto em si (execuções de `fastest generate` durante o desenvolvimento e dogfooding). Esses custos são marginais, estimados em menos de \$1.00 no total, dado o uso de modelos econômicos (gpt-4o-mini a ~\$0.002/chamada e claude-haiku a ~\$0.001/chamada).

7.2 Camada 2 -- Esforço Humano Real

Fase	Horas com IA	Atividades
Exposição	10 h	Documentação Sinfonia (4 artefatos), pesquisa de domínio, definição de personas
Composição	14 h	Design de prompts, arquitetura C4, canvas de

Ensaio	46 h	experimento, ideação de soluções
Ressonância	12 h	Implementação do CLI, 204 testes, CI/CD, documentação de riscos e lançamento
Total	82 h	Métricas de impacto, escalabilidade, feedback, apresentação, relatório final

7.3 Camada 3 -- Contrafactual Humano

Perfil	Valor/hora (BRL)	Fonte
Júnior	~R\$ 35	Glassdoor 2026 (médias de mercado)
Pleno	~R\$ 70	
Sênior	~R\$ 110	

Atividade	Perfil estimado	Horas estimadas sem IA	Custo estimado
Implementação do CLI (pipeline, providers, comandos)	Pleno/Sênior	132 h	R\$ 9.900
204 testes unitários e de comandos	Pleno	60 h	R\$ 4.500
Documentação Sinfonia (15 artefatos)	Sênior	52 h	R\$ 3.900
CI/CD setup (GitHub Actions, Node 18/20/22)	Pleno	18 h	R\$ 1.350
Total		262 h	R\$ 19.650

7.4 Análise Comparativa

Métrica	Valor
Custo total com IA (BRL)	R\$ 6.075 (R\$ 334,80 de IA + 82 h de supervisão a ~R\$ 70/h)
Custo total sem IA (BRL)	R\$ 19.650 (~ 262 h × R\$ 75/h médios)
Razão de economicidade	≈ 3,2×
Saving estimado (BRL)	≈ R\$ 13.575
Saving estimado (%)	≈ 69%

7.5 Limitações da Análise

A análise de economicidade apresenta limitações inerentes que devem ser consideradas na interpretação dos resultados:

1. **Contrafactual é estimativa subjetiva:** O cálculo de "horas sem IA" é uma projeção retrospectiva sujeita a viés de confirmação. O desenvolvedor tende a superestimar o tempo que levaria sem assistência, inflando artificialmente o saving.
2. **Curva de aprendizado não contabilizada:** O custo com IA não inclui o tempo investido em aprender a utilizar o Claude Code CLI de forma eficiente, configurar prompts e desenvolver fluência na co-pilotagem. Esse custo é real mas difícil de quantificar.
3. **Custo menor não implica qualidade equivalente:** Embora a IA tenha acelerado significativamente o desenvolvimento, revisão humana foi necessária em todo código gerado. Algumas saídas da IA continham erros sutis (imports incorretos, testes superficiais, mocks excessivos) que exigiram intervenção manual.
4. **Retrabalho por outputs incorretos:** Em diversas ocasiões, a IA gerou código que precisou ser parcialmente reescrito -- especialmente em áreas como resolução de imports, configuração de Jest com TypeScript e tratamento de edge cases em providers. Esse retrabalho está contabilizado nas horas de desenvolvimento mas é difícil de isolar.
5. **Horas de supervisão vs co-pilotagem:** É difícil separar o tempo gasto "supervisionando" a IA (revisando, corrigindo, redirecionando) do tempo gasto "co-pilotando" (ideando junto, iterando em soluções). Ambos contribuem para o resultado final mas representam esforços qualitativamente diferentes.
6. **Qualidade da documentação:** Os 15 artefatos da Sinfonia foram produzidos com assistência de IA, o que permitiu volume e consistência, mas a profundidade analítica em alguns artefatos pode diferir do que um especialista sênior produziria sem assistência.

8. Discussões Técnicas e Estratégicas

8.1 Factory Pattern para Multi-Provider

A decisão de utilizar o padrão Factory para abstrair provedores de LLM foi motivada pela necessidade de suportar OpenAI e Anthropic sem alterar o pipeline de geração. A ProviderFactory utiliza o prefixo do modelo (`claude-*`) para instanciar o provedor correto, implementando a interface `LLMProvider` com o método `generate()`.

Benefícios alcançados:

- Troca de provedor sem modificação no pipeline (`test-generator.service.ts` e `llm.service.ts` são agnósticos ao provedor)
- Adição de novos provedores (ex.: Ollama) requer apenas uma nova implementação da interface
- Testes unitários independentes por provedor (100% de cobertura em ambos)

8.2 Zero-shot vs RAG vs Fine-tuning

A estratégia de prompting zero-shot com contexto rico foi escolhida após avaliação de três alternativas:

Abordagem	Avaliação	Decisão
Zero-shot com contexto	O código-fonte do usuário fornece todo o contexto necessário. Simples, sem infraestrutura adicional.	Escolhida
RAG	Exigiria base de conhecimento de testes exemplares, vector store e pipeline de retrieval. Complexidade desnecessária para o escopo.	Descartada
Fine-tuning	Custo elevado, dados de treinamento insuficientes, perda de flexibilidade de trocar modelos.	Descartada

8.3 Guard Rails de Contexto

Os limites de contexto (max 20 arquivos, 4000 chars/arquivo, 30000 chars total) foram definidos empiricamente para equilibrar qualidade da geração e custo de tokens. Sem esses limites, projetos com dezenas de arquivos ou arquivos extensos excedem facilmente os limites de contexto dos modelos, gerando custos desnecessários e degradação da qualidade.

A flag `--strict-context` permite que equipes configurem o comportamento desejado: em modo estrito, violações de limites geram erro; no modo padrão, arquivos são truncados com marcador `/* ... truncated ... */`.

8.4 Retry com Feedback de Erro

A implementação de retry com re-envio de erros de compilação ao LLM representou a decisão técnica de maior impacto no projeto. Os resultados foram:

- **Sem retry:** Taxa de pass na primeira geração de ~80%
- **Com retry (ate 3 tentativas):** Taxa de pass de **92%+**

O mecanismo funciona enviando o código gerado que falhou junto com a mensagem de erro do TypeScript ou Jest de volta ao LLM, solicitando correção específica. O modelo "aprende" dos próprios erros no contexto da conversa e produz versões corrigidas com alta taxa de sucesso.

8.5 Trade-offs de Modelos

Modelo	Custo/chamada	Latência	Qualidade	Uso Recomendado
gpt-4o-mini	~\$0.002	Rápida	Boa	Uso diário, alto volume
gpt-4o	~\$0.02	Média	Muito boa	Código complexo

gpt-4	~\$0.06	Lenta	Excelente	Debugging, edge cases
claude-haiku	~\$0.001	Rápida	Boa	Custo mínimo
claude-sonnet	~\$0.015	Média	Muito boa	Equilíbrio custo/qualidade

A decisão padrão pelo gpt-4o-mini privilegia o cenário mais comum: uso frequente em desenvolvimento cotidiano, onde velocidade e custo são mais importantes que qualidade marginal.

8.6 Desafios Técnicos

1. **Correção de import paths:** O LLM não conhece a estrutura de diretórios do projeto e frequentemente gera imports com caminhos incorretos. O pós-processamento automático ajusta o primeiro `from '...'` para o caminho relativo correto, mas imports de dependências internas podem requerer a flag `--context`.
2. **Stripping de code fences:** Apesar da instrução explícita para retornar código sem blocos markdown, modelos frequentemente envolvem a resposta em ``typescript ... ``. A função `stripCodeFences()` remove esses blocos de forma robusta.
3. **Validação TypeScript antes do Jest:** Executar `tsc --noEmit` antes do Jest permite detectar erros de tipo antes da execução, economizando tempo e fornecendo mensagens de erro mais claras para o loop de retry.

9. Considerações Éticas

9.1 Vieses Identificados

Conforme a *Reflexão sobre o Uso de IA no Projeto*, os seguintes vieses foram identificados na geração de testes por LLMs:

1. **Viés de happy path:** LLMs tendem a gerar testes que validam o caminho feliz, negligenciando edge cases, cenários de erro e condições de contorno. Mitigação: instruções explícitas no prompt para cobrir cenários de erro + validação de cobertura.
2. **Viés linguístico:** Os prompts são escritos em português (pt-BR), o que pode afetar a qualidade da geração em comparação com prompts em inglês, dado que os modelos são predominantemente treinados em dados em inglês.
3. **Viés framework-centric:** Os testes gerados seguem fortemente as convenções de Jest, podendo não refletir boas práticas de outros frameworks (Mocha, Vitest). Mitigação: templates de prompt específicos por framework.
4. **Viés de padrão de código:** O LLM replica padrões de teste comuns do seu treinamento, que podem não ser ideais para o projeto específico do usuário.

9.2 Privacidade

- **Risco principal:** Código-fonte proprietário do usuário é transmitido para servidores da OpenAI ou Anthropic como parte do prompt.
- **Mitigações implementadas:**
 - Guard rails limitam a quantidade de código enviado
 - Modo `--dry-run` permite inspeção do prompt antes do envio
 - Chaves API armazenadas localmente com variáveis de ambiente como alternativa
 - Documentação clara sobre o fluxo de dados
- **Mitigação futura:** Suporte a modelos locais via Ollama eliminará a necessidade de transmissão externa.

9.3 Segurança

- **Prompt injection:** Risco baixo. O código-fonte é tratado como dados (não como instrução) no prompt. Porém, arquivos-fonte maliciosos poderiam teoricamente influenciar o comportamento do modelo.
- **Código gerado:** Risco muito baixo. O modo `--dry-run` permite revisão antes de salvar. Testes são executados em ambiente Jest isolado.
- **Dependências:** Auditoria regular de dependências npm recomendada. O projeto utiliza apenas dependências estabelecidas e bem mantidas.

9.4 Transparência

- **Provedor visível:** O provedor e modelo em uso são visíveis via `fastest config list` e podem ser alterados a qualquer momento.
- **Comando doctor:** `fastest doctor` exibe o estado completo da configuração, incluindo provedor ativo, fonte da API key e dependências.
- **Co-Authored-By:** Todos os commits gerados com assistência de IA incluem a tag `Co-Authored-By`, mantendo rastreabilidade completa.

9.5 Atribuição

- Claude foi utilizado como **par de programação**, não como substituto do desenvolvedor.
- Todas as decisões arquiteturais, de design e de produto foram tomadas pelo desenvolvedor humano.
- A IA acelerou a execução, mas não substituiu o julgamento técnico e crítico.
- O uso de modelos comerciais (OpenAI, Anthropic) implica em aceitar seus termos de serviço.
- A responsabilidade final pela qualidade do software e dos testes é do desenvolvedor.

10. Lições Aprendidas e Reflexões Finais

10.1 Reflexões sobre a Sinfonia

- **Exposição foi a fase mais rápida:** O problema era claro e bem definido desde o início (teste de software é negligenciado, IA pode ajudar). A identificação do domínio e a definição de personas fluíram naturalmente.
- **Composição foi a mais desafiadora:** O design de prompts requer iteração constante. A qualidade do output do LLM é altamente sensível a pequenas mudanças no prompt, exigindo experimentação sistemática.
- **Ensaio foi a fase mais volumosa:** A implementação do pipeline completo (providers, comandos, serviços, testes, CI) representou o maior esforço do projeto. A fase de testes, ironicamente, demandou significativo investimento apesar da ferramenta ser sobre geração de testes.
- **Ressonância foi a mais reflexiva:** Avaliar o impacto do projeto e coletar feedback forçou uma análise crítica sobre o que realmente funciona vs o que parece funcionar.

10.2 Proposta de Valor

O Fastest CLI demonstrou que é possível reduzir o tempo de criação de suítes de teste de **45-90 minutos para 2-5 minutos**, uma redução de 90-97%. Essa economia se traduz em 6.5-14 horas mensais por desenvolvedor (assumindo 10 suítes/mês), permitindo que equipes mantenham alta cobertura sem sacrificar velocidade de entrega.

10.3 Melhorias Futuras

1. **Suporte a Ollama (modelos locais):** Eliminar custos de API e preocupações com privacidade, permitindo uso offline.
2. **GitHub Action oficial:** Integração nativa com pipelines de CI/CD para geração automática de testes em PRs.
3. **Plugin VS Code:** Experiência integrada no editor, com geração de testes a partir do contexto visual.
4. **Suporte a mais frameworks:** Mocha, Node.js test runner nativo, Testing Library.
5. **Análise de qualidade de asserções:** Ir além de cobertura numérica, avaliando a profundidade e relevância das asserções geradas.

10.4 Aprendizados sobre IA

1. **IA generativa funciona muito bem para código estruturado:** Testes Jest seguem padrões previsíveis (`describe/it/expect`) que modelos geram com boa qualidade. A estrutura repetitiva é uma vantagem, não uma limitação.
2. **Retry é o game-changer:** Re-enviar erros de compilação ao LLM para auto-correção aumentou a taxa de sucesso de ~80% para 92%+. O modelo "aprende" dos próprios erros no contexto da conversa. Essa funcionalidade deveria ter sido implementada desde a v1.0.

3. **Validação em pipeline é essencial:** Sem compilação TypeScript + execução Jest + análise de cobertura, a geração de testes seria uma caixa-preta. O pipeline transforma output de IA em output confiável.
4. **Guard rails são necessários:** Sem limites de contexto, é fácil exceder limites de tokens e gerar custos desnecessários.
5. **IA como co-piloto, não substituto:** A ferramenta acelera drasticamente a escrita de testes, mas a revisão humana continua essencial. O melhor resultado vem da combinação: velocidade da IA + julgamento do desenvolvedor.

10.5 Relato Individual

Jorge Freitas (jlcf): Participar deste projeto foi uma experiência bastante enriquecedora, tanto do ponto de vista técnico quanto da organização do trabalho. Ao longo do desenvolvimento do Fastest CLI, pude vivenciar na prática como a inteligência artificial generativa pode atuar como uma verdadeira parceira de programação, acelerando tarefas repetitivas e ajudando a explorar soluções que talvez levassem muito mais tempo para serem alcançadas de forma manual. Ao mesmo tempo, aprendi que a IA não substitui o julgamento técnico do desenvolvedor: foi necessário revisar criticamente cada saída gerada, corrigir detalhes e tomar as decisões de arquitetura e de produto de forma consciente. A Metodologia Sinfonia teve um papel central nesse processo, pois estruturou o trabalho em movimentos bem definidos, da exposição do problema até a medição de impacto, o que trouxe clareza sobre o que precisava ser feito em cada etapa e evitou que o desenvolvimento perdesse o foco. Como reflexão final, acredito que a engenharia de software assistida por IA representa uma mudança importante na forma como construímos software: ela amplia a produtividade e democratiza o acesso a boas práticas, mas exige do profissional uma postura ainda mais crítica e responsável. Saio desta disciplina convencido de que dominar essa colaboração entre humano e IA será uma competência cada vez mais essencial no futuro da área.

11. Referências

1. SWEBOK v4.0 -- Software Engineering Body of Knowledge. IEEE Computer Society, 2024. Chapter 4: Software Testing. Disponível em: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
2. GARCIA, F.; MEDEIROS, A. **Metodologia Sinfonia: Framework Ágil para Desenvolvimento de Software Assistido por IA.** Universidade Federal de Pernambuco, Centro de Informática, 2025.
3. BROWN, S. **The C4 Model for Software Architecture.** Disponível em: <https://c4model.com>
4. OpenAI. **API Documentation.** Disponível em: <https://platform.openai.com/docs>

5. Anthropic. **API Documentation**. Disponível em: <https://docs.anthropic.com>
6. Jest. **Documentation**. Disponível em: <https://jestjs.io/docs>
7. commander.js -- Complete solution for node.js command-line interfaces.
Disponível em: <https://github.com/tj/commander.js>
8. Istanbul. **Code Coverage Tool**. Integrado ao Jest via nyc/Istanbul.
9. Vitest. **A Vite-native testing framework**. Disponível em: <https://vitest.dev>
10. chalk -- Terminal string styling for Node.js. Disponível em:
<https://github.com/chalk/chalk>

12. Apêndices

Apêndice A: Workflow Document

- docs/workflow-document.md

Apêndice B: Artefatos Sinfonia Completos

Os 15 artefatos da metodologia Sinfonia estão disponíveis integralmente no repositório, organizados em quatro diretórios:

Movimento 1 -- Exposição:

- docs/sinfonia/01_exposicao/01_domain_identification_canvas.md
- docs/sinfonia/01_exposicao/02_persona_model_canvas.md
- docs/sinfonia/01_exposicao/03_data_source_mapping.md
- docs/sinfonia/01_exposicao/04_strategy_action_canvas.md

Movimento 2 -- Composição:

- docs/sinfonia/02_composicao/01_prompt_design_record.md
- docs/sinfonia/02_composicao/02_solution_ideation_canvas.md
- docs/sinfonia/02_composicao/03_experiment_design_canvas.md

Movimento 3 -- Ensaio:

- docs/sinfonia/03_ensaio/01_c4_model.md
- docs/sinfonia/03_ensaio/02_intelligence_strategy_record.md
- docs/sinfonia/03_ensaio/03_risk_defensibility_checklist.md
- docs/sinfonia/03_ensaio/04_testing_validation_canvas.md
- docs/sinfonia/03_ensaio/05_launch_checklist.md

Movimento 4 -- Ressonância:

- docs/sinfonia/04_ressonancia/01_scale_impact_metrics.md
- docs/sinfonia/04_ressonancia/02_scalability_planning.md
- docs/sinfonia/04_ressonancia/03_feedback_insights_panel.md

Apêndice C: Catálogo de Prompts

O catálogo completo de prompts, incluindo templates, parâmetros, guard rails e pós-processamento, está documentado em:

- docs/sinfonia/02_composicao/01_prompt_design_record.md

Apêndice D: Evidencias

D.1 Resultado dos Testes (execução em 28/06/2026)

```
Test Suites: 14 passed, 14 total
Tests:       204 passed, 204 total
Snapshots:   0 total
Time:        7.147 s
Ran all test suites.
```

D.2 Histórico de Commits

```
d67dbaf fix: resolve merge conflicts with main and fix demo slide in roteiro
4371b46 feat: add use-case test type, interactive mode, and fix versioning to 0.1.0
2c53af6 docs: expand consolidated doc and AI reflection with full content
2680dc1 Claude/confident allen 2mb41n (#5)
ab3eb4d docs: add presentation documents for Sinfonia final presentation
fb4a1a9 feat: add Vitest support, LLM caching, init wizard, changelog (v2.1.0)
5199fca feat: add LLM retry logic, batch mode, and input validation
077b329 improve test coverage, remove unnecessary deps, add CI workflow
b5877f2 docs: add comprehensive sinfonia documentation for Fastest CLI v2.0.0 (#4)
58562ea docs: add complete Sinfonia methodology documentation (15 artifacts)
1d85530 chore: update package-lock.json after npm install
0ca91dc Merge pull request #3
44b70a1 feat: add integration test generation mode and coverage flow hints
9e80c63 chore: bump version to 2.0.0, add DEMO.md presentation script
a477a14 feat: Fastest CLI v2.0 – complete pipeline with streaming, multi-provider
f7d8c5e test: add provider tests – factory, OpenAI, Anthropic (142 total)
05f4e4b feat: streaming LLM output + multi-provider support
1c2b49a feat: JS/TS auto-detection, tsc validation, README install-first
8df7044 feat: before/after coverage delta and CLI command test coverage
0f596a0 feat(config): add config command to manage API key globally
51f2370 feat: support global CLI install from GitHub and npm registry
ddec494 feat(setup): guided first-time setup script and improved doctor checks
8c5dc7d feat: visual output, unit tests, order.service example, docs
2a2c98a feat(cli): add doctor command; improve test generation import handling
b85e5a3 feat: implement Fastest CLI – test generation pipeline from cards
7c3b01f initial commit
9fa0837 Initial commit
```

D.3 CI com GitHub Actions

Workflow configurado em `.github/workflows/` para execução automatizada de testes em Node.js 18, 20 e 22. Todas as builds passam com 204 testes e 0 falhas.

D.4 Link do PR #4

<https://github.com/jorgelcff/Fastest-CLI/pull/4> -- Documentação Sinfonia completa (15 artefatos).

D.5 Link do PR #5

<https://github.com/jorgelcff/Fastest-CLI/pull/5> -- Funcionalidades v0.1.0 (batch, retry, Vitest, cache, init).

Relatório gerado em 28 de Junho de 2026. Fastest CLI v0.1.0 -- 204 testes, 14 suites, 93%+ cobertura.